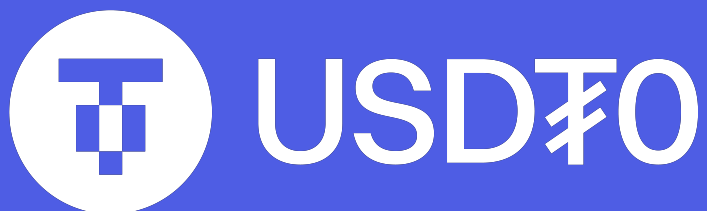


Hedera Token Service Connector Audit



October 28, 2025

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	5
Privileged Roles	6
Low Severity	7
L-01 Pragma Statements Span Across Several Minor Versions of Solidity	7
L-02 Lack of Storage Gap in Upgradeable Contracts	7
L-03 Missing Or Incomplete Docstrings	8
L-04 Unsafe ABI Encoding	8
L-05 Invalid serialNumbers Forwarding Possible in burnTokenPublic	9
L-06 Unused Functions With Internal or Private Visibility	9
L-07 Floating Pragma	10
L-08 Experimental Pragma	10
L-09 Token Key Creation May Fail Silently	11
L-10 bytesToAddress May Incorrectly Decode Provided Bytes	11
Notes & Additional Information	12
N-01 Multiple Possible Violations of 'No Conflicting Names' EEA Ethtrust Security Level [S] Requirement	12
N-02 Missing Zero-Address Check	13
N-03 Use calldata Instead of memory	13
N-04 Constants Not Using UPPER_CASE Format	14
N-05 Duplicate Imports	14
N-06 Function Visibility Overly Permissive	14
N-07 Unused State Variable	15
N-08 Use of uint/int Instead of uint256/int256	15
N-09 Multiple Optimizable State Reads	15
N-10 Unused Event	16
Client Reported	16
CR-01 Incorrectly Initialized Decimals	16
Conclusion	17

Summary

Type	Stablecoins	Total Issues	21 (13 resolved, 2 partially resolved)
Timeline	From 2025-10-06 To 2025-10-14	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	10 (7 resolved, 1 partially resolved)
		Notes & Additional Information	10 (5 resolved, 1 partially resolved)
		Client Reported Issues	1 (1 resolved)

Scope

OpenZeppelin performed an audit of the [Everdawn-Labs/usdt0-oft-contracts](#) repository at commit [c2c4660](#).

In scope were the following files:

```
contracts
├── hts
│   ├── HederaTokenService.sol
│   ├── IHederaTokenService.sol
│   ├── IHRC719.sol
│   ├── KeyHelper.sol
│   └── KeyHelperUpgradeable.sol
├── HTSConnectorUpgradeable.sol
└── IHRC755ScheduleFacade.sol
```

System Overview

The in-scope contracts allow for deploying the cross-chain USDT0 token on the Hedera network. In particular, the [HTSConnectorUpgradeable](#) contract serves as a connection between the cross-chain capabilities offered by the LayerZero OFT implementation and the native tokenization model which is specific to the Hedera network (called the Hedera Token Service).

The [Hedera Token Service](#) (HTS) allows for the creation and management of fungible and non-fungible tokens at the protocol level (no user-deployed ERC-20/721 needed). Tokens are configured with keys (admin, supply, KYC, freeze, etc.) and can be [created/updated](#) via Hedera APIs or from Solidity through the HTS system contract at [0x167](#), which exposes create/mint/burn/transfer/associate and other operations. This reduces contract surface area while retaining throughput and predictable fees. In this hybrid design, applications keep complex business logic in EVM smart contracts but delegate token actions to HTS via the precompile.

For familiar tooling, Hedera also offers ERC-20 and ERC-721 compatibility. So, standard calls (e.g., [transfer](#), [approve](#), [transferFrom](#)) [map](#) to HTS behavior. Off-chain services typically query history and events from [mirror nodes](#) rather than consensus nodes, fitting neatly into existing indexing/analytics stacks. Ultimately, the smart contracts under audit leverage this native tokenization ability to create a connector contract which manages critical aspects like minting and burning which are necessary for cross-chain transfers when using LayerZero's OFT service.

Security Model and Trust Assumptions

During the audit, the following trust assumptions were identified:

- Deployment and initialization must happen in the same transaction as the initial setup happens through an [initialize](#) call and [HTSConnectorUpgradeable](#) can be seized if left uninitialized.

- The Hedera Token Standard allows for the specification of keys with specific admin privileges. In particular, the `ADMIN`, `FREEZE`, `WIPE`, and `SUPPLY` keys are set within `HTSConnectorUpgradeable`. The `_adminKey` address has the ability to perform update/delete operations on the token, while the `_safeWallet` address has the ability to freeze and wipe token balances. Furthermore, the `HTSConnectorUpgradeable` contract has the ability to mint/burn tokens through its respective functions `_credit`, `mintTokenPublic`, and `_debit`, `burnTokenPublic`.

Privileged Roles

The `HTSConnectorUpgradeable` has the following privileged roles:

- **owner**: The holder of this role is set as the owner of the contract during initialization and has the ability to call `mintTokenPublic` as well as ownership admin functions inherited from `OwnableUpgradeable`.
- **delegate**: The holder of this role has the ability to make OApp configurations inside the endpoint.

Low Severity

L-01 Pragma Statements Span Across Several Minor Versions of Solidity

Using a pragma statement that spans several minor Solidity versions can lead to unpredictable behavior due to differences in features, bug fixes, deprecation, and compatibility between minor versions.

Throughout the codebase, multiple instances of pragma versions spanning across several minor versions of Solidity were identified:

- The pragma version `>=0.4.9 <0.9.0` used in [IHRC755ScheduleFacade.sol](#)
- The pragma version `>=0.5.0 <0.9.0` used in [HederaTokenService.sol](#)
- The pragma version `>=0.4.9 <0.9.0` used in [IHRC719.sol](#)
- The pragma version `>=0.4.9 <0.9.0` used in [IHederaTokenService.sol](#)
- The pragma version `>=0.5.0 <0.9.0` used in [KeyHelper.sol](#)
- The pragma version `>=0.5.0 <0.9.0` used in [KeyHelperUpgradeable.sol](#)

Consider pinning the Solidity version more specifically throughout the codebase to ensure predictable behavior and maintain compatibility across various compilers.

Update: Resolved in commit [e0563df](#). Now, the files only use Solidity versions starting from `0.8.0` up to, but not including, `0.9.0`.

L-02 Lack of Storage Gap in Upgradeable Contracts

The [HTSConnectorUpgradeable](#) and [KeyHelperUpgradeable](#) contracts are both intended to be used within an upgradeable context. However, neither of these contracts implements storage gaps to allow for the safe addition of state variables in future upgrades.

To allow the addition of new state variables without compromising storage compatibility with existing deployments, consider [leaving a storage gap](#) at the end of each contract.

Update: Resolved in [pull request #147](#). The Everdawn team stated:

| We have changed the format to comply with ERC-7201.

L-03 Missing Or Incomplete Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

- In `HTSConnectorUpgradeable.sol`, the `htsTokenAddress` state variable
- In `HTSConnectorUpgradeable.sol`, the `MintedToken` event
- In `HTSConnectorUpgradeable.sol`, the `BurnedToken` event
- In `HTSConnectorUpgradeable.sol`, the `TokenCreated` event
- In `IHRC755ScheduleFacade.sol`, the `IHRC755ScheduleFacade` interface
- In `HederaTokenService.sol`, the `HederaTokenService` abstract contract
- In `IHRC719.sol`, the `IHRC719` interface
- In `IHederaTokenService.sol`, the `IHederaTokenService` interface
- In `KeyHelper.sol`, the `KeyHelper` abstract contract
- In `KeyHelper.sol`, the `Bits` library
- In `KeyHelperUpgradeable.sol`, the `KeyHelperUpgradeable` abstract contract
- In `KeyHelperUpgradeable.sol`, the `Bits` library

Note that in both the `HederaTokenService` and the `IHederaTokenService.sol` files, the `transferToken` function does not document the `responseCode` return variable.

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Acknowledged, not resolved.

L-04 Unsafe ABI Encoding

It is not an uncommon practice to use `abi.encodeWithSignature` or `abi.encodeWithSelector` to generate calldata for a low-level call. However, the first option is not typo-safe and the second option is not type-safe. The result is that both of these methods are error-prone and should be considered unsafe.

Within `HederaTokenService.sol`, multiple instances of unsafe ABI encoding were identified:

- The use of `abi.encodeWithSelector` within the `mintToken` function

- The use of `abi.encodeWithSelector` within the `burnToken` function
- The use of `abi.encodeWithSelector` within the `createFungibleToken` function
- The use of `abi.encodeWithSelector` within the `transferToken` function
- The use of `abi.encodeWithSelector` within the `updateTokenKeys` function

Consider replacing all the occurrences of unsafe ABI encodings with `abi.encodeCall` which checks whether the supplied values actually match the types expected by the called function and also avoids errors caused by typos.

Update: Resolved at commit [c2ce348](#).

L-05 Invalid `serialNumbers` Forwarding Possible in `burnTokenPublic`

The `HTSConnectorUpgradeable` contract's `burnTokenPublic` contract requires an empty `serialNumbers` array. Nevertheless, `burnTokenPublic` exposes a `serialNumbers` parameter and forwards it directly to the precompile. Supplying any non-empty array makes the precompile return an error code, leading to a revert (`HTSBurnFailed`).

Consider removing the `serialNumbers` parameter and always supplying an empty array to the `burnToken` call.

Update: Resolved at commit [31c8072](#).

L-06 Unused Functions With Internal or Private Visibility

Throughout the codebase, multiple instances of unused functions were identified:

- The `updateTokenKeys` function in `HederaTokenService.sol`
- The `getSingleKey` function in `KeyHelper.sol`
- The `getSingleKey` function in `KeyHelper.sol`

To improve the overall clarity, intentionality, and readability of the codebase, consider using or removing any currently unused functions.

Update: Partially resolved. The Everdawn team stated:

| Acknowledged, the `KeyHelper.sol` contract has been removed.

L-07 Floating Pragma

Pragma directives should be fixed to clearly identify the Solidity version with which the contracts will be compiled.

Throughout the codebase, multiple instances of floating pragma directives were identified:

- `HTSConnectorUpgradeable.sol` has the `solidity ^0.8.22` floating pragma directive.
- `IHRC755ScheduleFacade.sol` has the `solidity >=0.4.9 <0.9.0` floating pragma directive.
- `HederaTokenService.sol` has the `solidity >=0.5.0 <0.9.0` floating pragma directive.
- `IHRC719.sol` has the `solidity >=0.4.9 <0.9.0` floating pragma directive.
- `IHederaTokenService.sol` has the `solidity >=0.4.9 <0.9.0` floating pragma directive.
- `KeyHelper.sol` has the `solidity >=0.5.0 <0.9.0` floating pragma directive.
- `KeyHelperUpgradeable.sol` has the `solidity >=0.5.0 <0.9.0` floating pragma directive.

Consider using fixed pragma directives. Fixing the pragma directive to a more recent, stable version will avoid exposing the system to known compiler bugs from previous versions.

Update: Acknowledged, not resolved.

L-08 Experimental Pragma

The experimental pragma unlocks non-default features, but lacks stability guarantees between non-breaking releases, making it unsuitable for production.

Throughout the codebase, multiple instances of experimental pragmas were identified:

- The file `IHRC755ScheduleFacade.sol` has the `experimental ABIEncoderV2` experimental pragma.
- The file `HederaTokenService.sol` has the `experimental ABIEncoderV2` experimental pragma.
- The file `IHederaTokenService.sol` has the `experimental ABIEncoderV2` experimental pragma.
- The file `KeyHelper.sol` has the `experimental ABIEncoderV2` experimental pragma.

- The file `KeyHelperUpgradeable.sol` has the `experimental ABIEncoderV2` experimental pragma.

Consider avoiding the use of experimental pragmas within code that is meant for production deployments.

Update: Resolved at commit [e0563df](#) and at commit [738ac24](#).

L-09 Token Key Creation May Fail Silently

The `KeyHelperUpgradeable.sol` contract's `getKeyValueTypeForContract` function is intended to translate a caller-supplied `keyValueType` enum into the corresponding Hedera `KeyValue`. The issue is that only three enum members – `INHERIT_ACCOUNT_KEY`, `CONTRACT_ID`, and `DELEGATABLE_CONTRACT_ID` – are handled explicitly in this variant of `getSingleKey`. For every other *still-valid* enum value (e.g. `ED25519` and `SECP256K1`), the function falls through the `if/else`-chain, nothing is written to the return variables, and no revert is raised. External contracts that build a token key out of user-provided data may therefore end up with a struct whose `keyType` bit (ADMIN, SUPPLY, WIPE) is set while the accompanying `KeyValue` is empty. This may result in ineffective or faulty validation on sensitive token operations.

Consider adding a revert to prevent the accidental use of the `getSingleKey(KeyType,KeyValueType,address)` function for `KeyValueType` types which are not intended for use with `address` types.

Update: Resolved at commit [63d70dc](#).

L-10 `bytesToAddress` May Incorrectly Decode Provided Bytes

The `KeyHelperUpgradeable` contract's `bytesToAddress` function converts a provided `bytes` parameter to an `address` type. It achieves this by retaining the first 20 bytes of the provided value. While this is acceptable if the provided values are encoded using `abi.encodePacked(address)`, the conversion may still fail for addresses encoded in 32-byte words – such as when `abi.encode(address)` is used – due to the lower bits containing the address information. Subsequently, the `TokenKey`'s `KeyValue` may be stored incorrectly, creating a key which the user would be unable to use.

Consider revising the `bytesToAddress` conversion function to capture the lower 20 bytes of the provided input. In addition, consider documenting the expected behavior within the contract.

Update: Resolved in [pull request #144](#) at commit [ccb35f8](#). The Everdawn team stated:

We have resolved this issue by removing the `bytesToAddress` function and explicitly reverting in the case of invalid `KeyValueType` inputs.

Notes & Additional Information

N-01 Multiple Possible Violations of 'No Conflicting Names' EEA Ethtrust Security Level [S] Requirement

A possible violation of [EEA EthTrust Security Level \[S\]](#) 'No Conflicting Names' requirement was identified. This requirement demands that, to eliminate confusion, any function or variable name will not be reused. However, it does allow functions to be overridden (e.g. from a Base contract) so long as there is only one version of the function that operates within the code.

Throughout the codebase, the `getSingleKey` name is used for two different functions which perform similar key-related tasks.

- `getSingleKey` within the `KeyHelper` contract
- `getSingleKey` within the `KeyHelper` contract
- `getSingleKey` within the `KeyHelperUpgradeable` contract
- `getSingleKey` within the `KeyHelperUpgradeable` contract

Consider addressing any instances of conflicting/reused identifiers.

Update: Partially resolved at commit [940ed90](#). The `KeyHelperUpgradeable` contract still contains two functions named `getSingleKey`.

N-02 Missing Zero-Address Check

When operations with address parameters are performed, it's crucial to ensure the address is not set to zero. Setting an address to zero is problematic because it has special burn/renounce semantics. This action should be handled by a separate function to prevent accidental loss of access during value or ownership transfers.

Within `HTSConnectorUpgradeable.sol`, the `_safeWalletAddr` operation is missing a zero-address check.

Consider always performing a zero-address check before assigning a state variable.

Update: *Acknowledged, not resolved.*

N-03 Use `calldata` Instead of `memory`

When dealing with the parameters of `external` functions, it is more gas-efficient to read their arguments directly from `calldata` instead of storing them to `memory`. `calldata` is a read-only region of memory that contains the arguments of incoming `external` function calls. This makes using `calldata` as the data location for such parameters cheaper and more efficient compared to `memory`. Thus, using `calldata` in such situations will generally save gas and improve the performance of a smart contract.

Within `HTSConnectorUpgradeable.sol`, multiple instances where function parameters should use `calldata` instead of `memory` were identified:

- The `_name` parameter
- The `_symbol` parameter
- The `adminKey` parameter

Consider using `calldata` as the data location for the parameters of `external` functions to optimize gas usage.

Update: *Acknowledged, not resolved.*

N-04 Constants Not Using UPPER_CASE Format

Throughout the codebase, multiple instances of constants not being declared using the UPPER_CASE format were identified:

- The `_decimals` constant declared in [line 39 of HTSConnectorUpgradeable.sol](#)
- The `precompileAddress` constant declared in [line 12 of HederaTokenService.sol](#)
- The `defaultAutoRenewPeriod` constant declared in [line 14 of HederaTokenService.sol](#)

According to the [Solidity Style Guide](#), constants should be named with all capital letters with underscores separating words. For improved readability, consider following this convention.

Update: Resolved at commit [e0563df](#).

N-05 Duplicate Imports

In [HTSConnectorUpgradeable.sol](#), there are multiple instances of duplicate imports as the dependencies have already been imported through the [KeyHelperUpgradeable](#) contract, which is imported into the [HTSConnectorUpgradeable](#). Specifically:

- Import [import "../hts/HederaTokenService.sol";](#)
- Import [import "../hts/IHederaTokenService.sol";](#)

Consider removing any duplicate imports to improve the overall clarity and readability of the codebase.

Update: Resolved at commit [9a7a053](#).

N-06 Function Visibility Overly Permissive

The `token` function in [HTSConnectorUpgradeable.sol](#) with `public` visibility could be limited to `external`.

To better convey the intended use of functions and to potentially realize some additional gas savings, consider changing a function's visibility to be only as permissive as required.

Update: Acknowledged, not resolved.

N-07 Unused State Variable

In `KeyHelperUpgradeable.sol`, the `supplyContract` state variable is unused.

To improve the overall clarity and intent of the codebase, consider removing the unused state variable.

Update: Resolved. The Everdawn team stated:

As a result of the changes made to `bytesToAddress`, we believe that this issue is not relevant anymore.

N-08 Use of `uint/int` Instead of `uint256/int256`

`uint/int` is an alias for `uint256/int256`. However, for clarity and consistency, it is recommended to use `uint256/int256` explicitly. Throughout the codebase, there multiple instances of `uint/int` being used instead of `uint256/int256` were identified:

- The `int` in `HTSConnectorUpgradeable.sol`
- The `int` in `HederaTokenService.sol`
- The `int` in `HederaTokenService.sol`
- The `int` in `HederaTokenService.sol`
- The `int` in `HederaTokenService.sol`
- The `uint` in `IHederaTokenService.sol`

In favor of explicitness, consider replacing all instances of `int/uint` with `int256/uint256`.

Update: Resolved at commit [a17febb](#).

N-09 Multiple Optimizable State Reads

Throughout the codebase, multiple instances of optimizable storage reads were identified:

- The `_safeWallet` storage read in `HTSConnectorUpgradeable.sol`
- The `htsTokenAddress` storage read in `HTSConnectorUpgradeable.sol`
- The `htsTokenAddress` storage read in `HTSConnectorUpgradeable.sol`
- The `htsTokenAddress` storage read in `HTSConnectorUpgradeable.sol`
- The `htsTokenAddress` storage read in `HTSConnectorUpgradeable.sol`

- The `supplyContract` storage read in `KeyHelper.sol`.

Consider reducing SLOAD operations that consume unnecessary amounts of gas by caching the values in a memory variable.

Update: Acknowledged, not resolved.

N-10 Unused Event

In `HederaTokenService.sol`, the `CallResponseEvent` event is unused.

To improve the overall clarity, intentionality, and readability of the codebase, consider emitting or removing any currently unused events.

Update: Resolved at commit [20f616c](#).

Client Reported

CR-01 Incorrectly Initialized Decimals

The `HTSConnectorUpgradeable` contract's `constructor` initializes the `OFTCoreUpgradeable` parent contract with a local `decimals` value of 8. However, the `HTSConnectorUpgradeable` contract subsequently creates an HTS token with only `6 decimals`. Consequently, the bridging arithmetic inside LayerZero's OFT logic expects the local token to have 8 decimals, whereas in reality, the local token would only have 6 decimals. This breaks accounting during bridging operations as bridged amounts could be over- or understated by two orders of magnitude.

Considering making the `decimals` value a constant and using this constant value for both the underlying Hedera Token Service token creation and the LayerZero initialization.

This issue was identified during the audit of the in-scope contracts. However, upon further investigation, it was observed that the Everdawn team had already created a PR with a fix for this issue.

Update: Resolved in commit [213d154](#).

Conclusion

The reviewed USDT0 contracts enable the use of LayerZero OFT functionality on the Hedera network through a Hedera Token Service (HTS) connector contract. The HTS contract enables cross-chain minting and burning through integration with the native HTS service. Overall, the codebase was found to be well implemented, with the bulk of the logic residing within the existing HTS functionality.

The Everdawn team is appreciated for being responsive throughout the audit, providing valuable information regarding the in-scope codebase whenever requested.